Function expression

25 August 2026 18:54

In JavaScript a function is not a "Magical language structure", but a special kind of value.

The syntax that we used before is called Function Declaration:

```
function sayHi(){
   console.log(`Hello!`);
}
```

There is another syntax for creating a function that is called a function expression. It allow us to create a new function in the middle of any expression.

For example:

```
let sayHi = function(){
   console.log(`Hello!`);
};
```

Here we can see a variable *sayHi*, getting a value. The new function created as function(){console.log(`Hello`); };
As the function creation happens in the context of the assignment expression (to the right side of =) this is a function expression.

Please note there's no name after the function keyword. Omitting a name is allowed for function expression.

Here we immediately assign it to the variable, so the meaning of these code samples is the same: "create a function and put it into the variable sayHi".

In more advanced situations that we will come across later, a function may be created and immediately called or scheduled for a later execution, not stored anywhere thus remaining anonymous.

Function is a value

Let's reiterate. No matter how the function is created, a function is a value. Both examples above store a function in the, *sayHi* variable.

We can even print out that value using console.log():

```
function sayHi(){
   console.log(`hello!`);
}

console.log(sayHi); //method as value
sayHi(); //method call
```

Output:

```
[Function: sayHi]
hello!
```

Please note that the line before last line does not run the function because there are no parentheses after *sayHi*. There are programming languages where any mention of a function name causes its execution, but Javascript is not like that.

In Javascript, a function is a value, so we can deal with it as a value. The code above shows it's String representation which is the source code.

Is a special value in the sense that we can call it like sayHi().

But it's still a value, so we can work with it like with other kinds of values.

We can copy a function to another variable.

```
function sayHi() { //1. create
    console.log(`hello!`);
}
console.log(sayHi);
let func1 = sayHi; // 2. copy
func1(); //Hello 3 run the copy (it works)
sayHi(); //hello // this still works too
```

Output:

```
[Function: sayHi]
hello!
hello!
```

Here's what happens above in detail:

1. The function declaration (1) creates the function and puts it into the variable name sayHi.
2. Line (to) copies it into the variable func. Please note again: There are no parentheses after sayHi. If there were, then func = sayHi() would write the result of the call sayHi() into func, not the function sayHi itself.
3. Now the function can be called as both sayHi() and func().

We could also have used a function expression to declare, *sayHi* in the first line:

```
let sayHi = function(){ //1. create
   console.log();
};

let func = sayHi; //2. copy
//…
```

Everything would work the same

Why is there a semicolon at the end?

You might wonder why do function expression have a; at the end but function declaration do not:

```
function sayHi(){
  //…
}

let sayHi = function(){
  //…
};
```

The answer is simple: A function expression is created here as function(…) {...} inside the assignment statement: let sayHi = …;. The semicolon ; is recommended at the end of the statement, it's not the part of the function syntax.
The semicolon ; would be there for a simple assignment such as let sayHi= 5;, and it's also there for a function assignment.

Callback functions
Let's look at the more examples of passing functions as values and using function expression. We will write a function *ask(question, yes no)* With three parameters:
question
Text of the question
yes
Function to run if the answer is "yes".
no
Function to run if the answer is "no".

The function should ask the question and depending on the users answers call yes() or no():

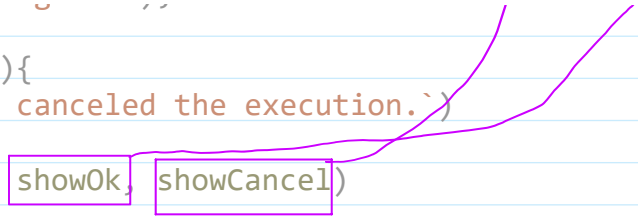
```
const prompt = require('prompt-sync')();
function ask(question, yes, no) {
    let q = prompt(`${question}`)
    if (q === `yes`) {
        yes();
    }
    else {
        no();
    }
}
function showOk() {
    console.log(`You agreed`);
}
function showCancel(){
```

yes = showOk

no = showCancel

This is due to Function Expression

```
}
function showCancel(){
    console.log(`You canceled the execution.`)
}
ask(`Do you agree?`, showOk, showCancel)
```



Output:

1) Do you agree?yes
   You agreed

2) Do you agree?no
   You canceled the execution.

In practice, such functions are quite useful. The major difference between a real-life ask and the example above is that real-life functions are more complex ways to interact with the user than a simple confirm. In the browser such function usually draw a nice - looking question window. But that's another story.

The arguments showOk and showCanel of ask are called **callback** function or just **callbacks.**

The idea is that we pass a function and expect it to be called back later if necessary. In our case, **showOK** becomes the callback for **Yes** answer and **showCancel** for **No** answer.

A function is a value representing an "action"

The regular values like strings or numbers represents the data.
A function can be perceived as an action.
We can pass it between variables and run when we want.

Function Expression vs Function Declaration
Let's formulate the key difference between function declaration and expression.

First, the syntax: how to differentiate between them in the code?

- Function declaration: A function declared as a separate statement in the main code flow.
  - Function Declaration:

```
function sum(a, b){
  return a+b;
}
```

- Function expression: of function created inside an expression or inside another syntax construct. Here the function is created on the right side

of the *"assignment expression" =:*

```
let sum = function(){
   return a + b;
};
```

The more subtle difference is when a function is created by the Javascript engine.

A function expression is created when the execution reaches it. And is usable only from that moment.

Once the execution flow passes to the right side of the assignment, let sum = function()… - Here we go, the function is created and can be used (assigned, called, etc) from now on.

Function declarations are different.

A function declaration can be called earlier than it is defined.
For example, a global function declaration is visible in the whole script, no matter where it is.
That's due to internal algorithms. When Javascript prepares to run the script, it first looks for global function declarations in it and creates the function. We can think of it as an **"initialization stage"**.

And after all function declarations are processed, the code is executed, so it has access to these functions.
For example this works:

```
   sayHi("John");
   function sayHi(name){ <—— strict mode.
       console.log(`Hello, ${name}`)
   }
```

Output:
   Hello, John

The function declaration **sayHi** is created when JavaScript is preparing to start the script and is visible everywhere in it.

… If it were a function expression then it wouldn't work:

```
   sayHi("John");
   sayHi = function(name){ //(*)
       console.log(`Hello, ${name}`)
   }
```

Output:
   ReferenceError: sayHi is not defined

```
        at Object.<anonymous>
```

Function expressions are created when the execution reaches them. That would happen only in the line (*). Too late.

Another special feature of function declaration is their block scope.

In strict mode, when function declaration is within a code block, it's visible everywhere inside the block. Just not outside of it.

For instance, let's imagine that we need to declare a function welcome() depending on the age variable that we get during runtime. And then we plan to use it in sometime later.

If we use function declaration, it won't work as intended.

```
const prompt = require(`prompt-sync`)();
let age = prompt(`What is your age? `, 18);
console.log(age);
if(age < 18){
    function welcome(){
        console.log(`Hello!`);
    }
}
else{
    function welcome(){
        console.log(`Greetings`);
    }
}
//... use it later
welcome(); //no output
```

Output:

1. What is your age? 17
   17
   Hello!

2. What is your age? 20
   20
   Greetings

```
const prompt = require(`prompt-sync`)();
let age = prompt(`What is your age? `, 18);
console.log(age);
if(age < 18){
    welcome();
    function welcome(){
```

```
            console.log(`Hello!`);
        }
        welcome();
    }
    else{
        function welcome(){
            console.log(`Greetings`);
        }
    }
    //... use it later
    welcome();
```

Output:

1. What is your age? 18
   18
   Greetings

2. What is your age? 17
   17
   Hello!
   Hello!
   Hello! //this is from last line

When to choose function declaration versus function expression?

As a rule of thumb, when we need to declare a function, the first thing to consider is function declaration syntax. It gives more freedom in how to organize our code, because we can call such functions before they are declared.

That's also better for readability as its easier to look up function F(...) {... } in the code, then let F = function (…) {... }; Function declarations are more "**eye catching**".

… But if a function declaration does not suit us for some reason or we need a conditional declaration? (we have just seen an example) then function expression should be used.

Summary

- Functions are values. They can be assigned, copied, or declared in any place of the code.
- If the function is declared as a separate statement in the main code flow, that's called a "**function declaration**"
- Function declarations are processed before the code block is executed. They are visible everywhere in the block.
- Function expressions are created when the execution flow reaches them.

In most cases when we need to declare a function, a function declaration is preferable because it is visible prior to the declaration itself. That gives us more flexibility in code organization and is usually more readable.

So we should use a function expression only when a function declaration is not fit for the task. We have seen couple of examples of that.